

Convolution

- To achieve convolutions with stride!=1 we usually pair conv_with_stride_1 i.e. to prepend or append conv_with_stride_1 with either a downsampling or an upsampling operation. The reason is we can define the backpropagation operations clearly for both conv_with_stride_1 and each of the downsampling and upsampling operations with some upsampling or downsampling factors, S. Related: [Resampling](#)
- CNNs are **learnable filters**.
- Convolutional layer can be thought of as a 2D analog of linear layer for 1D vectors, although there are some subtleties like weight sharing and sliding window approach for memory to be tractable in processing images.
 - Note: in my opinion, resolution (width and height) of the image or feature maps are of secondary importance compared to the feature dimension of the feature maps since we're able to achieve the desired resolution of almost any sizes by means of for example Spatial Pooling Pyramid or using zero padding.
- Convolution formula $W_{out} = \frac{W_{in} + 2P - K}{stride} + 1$.
- Transposed convolution: $W_{out} = stride * (W_{in} - 1) + K - 2P$
- Tip: By padding the input image with $K//2$ the output will be of the same width and height with the input. example:
 - padded image size = $10 + 3//2 + 3//2 = 12$
 - convolve with kernel of size $K = 3$.
 - get original size back: $W_{out} = \frac{12+0-3}{1} + 1 = 9 + 1 = 10$
- Tip: By padding the output with $K - 1$, and convolving it with the kernel will return an feature map with the same size as the original input. This property is used in conv backprop, example:
 - Original input size: $W_{in} = 10$
 - kernel_size = 3
 - $W_{out} = \frac{10+0-3}{1} + 1 = 8$
 - now pad output with $K - 1$,
 - $W_{out_padded} = 8 + 2 + 2 = 12$
 - convolve resulting output with kernel size of K :
 - $\frac{12+0-3}{1} + 1 = 10$.
 - Since in backpropagation, $dLdx = dLdz \otimes flipped(W)$, where W is the filter.
- How big is our receptive field?
 - $r_0 = \sum_{l=1}^L ((k_l - 1) \prod_{i=1}^{l-1} s_i) + 1$
 - r_0 is receptive field size
 - k_l is kernel size at layer l .
 - $\sum_{l=1}^L$ sum over network layers.
 - $\prod_{i=1}^{l-1}$ is the product of strides up to layer l .
- 1D CNN
 - Using a stride of 1, since we have assumed separate treatment for convolutions with stride ($s > 1$) or ($0 < s < 1$), see: [Resampling](#), to be handled by downsampling or upsampling layer, respectively.
 - Forward
 - summing operations: red: collapsing of axes with the same dimensions, blue: convolution.
 - $$x \otimes W = z$$
 - Forward

$$(N, C_{in}, W_{in}) \otimes (C_{out}, C_{in}, K) = (N, C_{out}, W_{out})$$
 - $$W_{out} = \frac{W_{in} - K}{1} + 1$$
 - Backward:
 - To get $dLdx$ after padding $dLdz$ with $K - 1$ zeros left and right, we will convolve $dLdz$ with the 180° rotated or (fliplr, flipud) in numpy of filter W .
 - Size of $dLdz$ after padding: $W_{out} = W_{in} + K - 1$
 - $$dLdz \otimes flipped(W) = dLdx$$

$$4. \quad (N, C_{out}, W_{out}) \otimes (C_{out}, C_{in}, K) = (N, C_{in}, W_{in})$$

5. After convolution we will recover back a tensor of the size of the input x , i.e. i.e. $\frac{W_{in}+K-1-K}{1} + 1 = W_{in}$. Therefore, we will be using 'full' option during the convolution to get $dLdx$.
6. To get $dLdW$, the formula is to convolve the input x with the output of the derivative. Convolve the input x with the derivative of the output, $dLdz$:

$$7. \quad X \otimes dLdz = dLdW$$

$$8. \quad (N, C_{in}, W_{in}) \otimes (N, C_{out}, W_{out}) = (C_{out}, C_{in}, K)$$

9. To validate this convolution operation, the output tensor is of size

$$\frac{W_{in} - (W_{in} - K + 1)}{1} + 1 = K$$

10. 2D CNN

1. Similar to 1D CNN we extend by adding a new H axis.
2. Backward

$$dLdz \otimes \text{flipped}(W) = dLdx$$

$$1. \quad (N, C_{out}, H_{out}, W_{out}) \otimes (C_{out}, C_{in}, K, K) = (N, C_{in}, H_{in}, W_{in})$$

$$X \otimes dLdz = dLdW$$

$$2. \quad (N, C_{in}, H_{in}, W_{in}) \otimes (N, C_{out}, H_{out}, W_{out}) = (C_{out}, C_{in}, K, K)$$

11. stride=1

Forward Pass

- Convolution between Input X and Filter F results in an output O as shown:

$$\begin{pmatrix} O_{11} & O_{12} \\ O_{21} & O_{22} \end{pmatrix} = \text{Convolution} \left(\begin{pmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{pmatrix}, \begin{pmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{pmatrix} \right)$$

Input X Filter F

1.

Carnegie Mellon University

Forward Pass

- Convolution between Input X and Filter F results in an output O as shown:

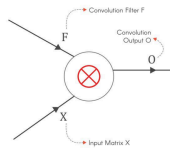
$$\begin{pmatrix} O_{11} & O_{12} & O_{13} \\ O_{21} & O_{22} & O_{23} \end{pmatrix} = \begin{pmatrix} X_{11}F_{11} & X_{12}F_{12} & X_{13}F_{13} \\ X_{21}F_{11} & X_{22}F_{12} & X_{23}F_{13} \\ X_{31}F_{11} & X_{32}F_{12} & X_{33}F_{13} \end{pmatrix}$$

Input X Filter F

Carnegie Mellon University

Computing Derivatives

- We want to calculate derivatives of output O w.r.t. Input X and Filter F



2.

Carnegie Mellon University

Derivative of Loss w.r.t Filter

- We can apply chain rule as follows

O: affine output of current layer

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial O} * \frac{\partial O}{\partial F}$$

For every element of F

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^M \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial F_i}$$

Carnegie Mellon University

Derivative of Loss w.r.t Filter

- We can expand the chain rule summation as follows:

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^M \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial F_i}$$

$$O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$$

$$O_{12} = X_{12}F_{11} + X_{13}F_{12} + X_{22}F_{21} + X_{23}F_{22}$$

$$O_{21} = X_{21}F_{11} + X_{22}F_{12} + X_{31}F_{21} + X_{32}F_{22}$$

$$O_{22} = X_{22}F_{11} + X_{23}F_{12} + X_{32}F_{21} + X_{33}F_{22}$$

3.

Carnegie Mellon University

Derivative of Loss w.r.t Filter

Forward Pass Equations

$$\begin{aligned} O_{11} &= X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22} \\ O_{12} &= X_{12}F_{11} + X_{13}F_{12} + X_{22}F_{21} + X_{23}F_{22} \\ O_{21} &= X_{21}F_{11} + X_{22}F_{12} + X_{31}F_{21} + X_{32}F_{22} \\ O_{22} &= X_{22}F_{11} + X_{23}F_{12} + X_{32}F_{21} + X_{33}F_{22} \end{aligned}$$

Backward Equations

$$\begin{aligned} \frac{\partial L}{\partial F_{11}} &= \frac{\partial L}{\partial O_{11}} * X_{11} + \frac{\partial L}{\partial O_{12}} * X_{12} + \frac{\partial L}{\partial O_{21}} * X_{21} + \frac{\partial L}{\partial O_{22}} * X_{22} \\ \frac{\partial L}{\partial F_{12}} &= \frac{\partial L}{\partial O_{11}} * X_{12} + \frac{\partial L}{\partial O_{12}} * X_{13} + \frac{\partial L}{\partial O_{21}} * X_{22} + \frac{\partial L}{\partial O_{22}} * X_{23} \\ \frac{\partial L}{\partial F_{21}} &= \frac{\partial L}{\partial O_{11}} * X_{21} + \frac{\partial L}{\partial O_{12}} * X_{22} + \frac{\partial L}{\partial O_{21}} * X_{31} + \frac{\partial L}{\partial O_{22}} * X_{32} \\ \frac{\partial L}{\partial F_{22}} &= \frac{\partial L}{\partial O_{11}} * X_{22} + \frac{\partial L}{\partial O_{12}} * X_{23} + \frac{\partial L}{\partial O_{21}} * X_{32} + \frac{\partial L}{\partial O_{22}} * X_{33} \end{aligned}$$

Carnegie Mellon University

Derivative of Loss w.r.t Input

If you look at this operation closely, it could be computed by convolving the input map with output affine derivative map

$$\begin{pmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{pmatrix} = \text{Convolution} \left(\begin{pmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{pmatrix}, \begin{pmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{pmatrix} \right)$$

Can be viewed as convolution of input and affine derivative

4.

Loss Derivative Equations

Carnegie Mellon University

Derivative of Loss w.r.t Input

- Simplifying for the local derivative we get

For every element of X_i

$$\frac{\partial L}{\partial X_i} = \sum_{k=1}^M \frac{\partial L}{\partial O_k} * \frac{\partial O_k}{\partial X_i}$$

Carnegie Mellon University

Derivative of Loss w.r.t Input

$$\begin{aligned} O_{11} &= X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22} & \frac{\partial L}{\partial O_{11}} &= \frac{\partial L}{\partial O_{11}} F_{11} + \frac{\partial L}{\partial O_{11}} F_{11} \\ O_{12} &= X_{12}F_{11} + X_{13}F_{12} + X_{22}F_{21} + X_{23}F_{22} & \frac{\partial L}{\partial O_{12}} &= \frac{\partial L}{\partial O_{12}} F_{12} + \frac{\partial L}{\partial O_{12}} F_{21} \\ O_{21} &= X_{21}F_{11} + X_{22}F_{12} + X_{31}F_{21} + X_{32}F_{22} & \frac{\partial L}{\partial O_{21}} &= \frac{\partial L}{\partial O_{21}} F_{11} + \frac{\partial L}{\partial O_{21}} F_{12} \\ O_{22} &= X_{22}F_{11} + X_{23}F_{12} + X_{32}F_{21} + X_{33}F_{22} & \frac{\partial L}{\partial O_{22}} &= \frac{\partial L}{\partial O_{22}} F_{12} + \frac{\partial L}{\partial O_{22}} F_{21} \end{aligned}$$

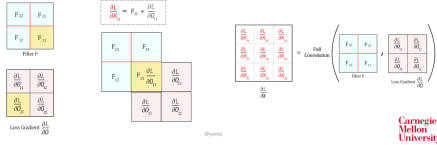
Forward Pass Equations

Backward Equations

5.

Derivative of Loss w.r.t Input

- Simplifying computation:**
 - Flip the Filter horizontally and vertically
 - Perform Full Convolution of flipped filter with the affine derivatives

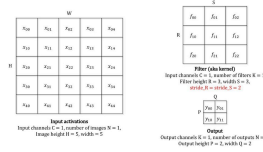


6.

12. stride > 1:

Backprop in CNN stride > 1

- Let us consider an example with **stride > 1** both in horizontal and vertical direction
- We have Input X and Filter as shown



1.

Forward Pass for stride > 1

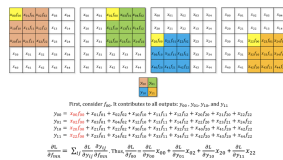
- Forward Pass Equations

$$\begin{aligned} y_{00} &= x_{00}f_{00} + x_{01}f_{01} + x_{02}f_{02} + x_{03}f_{03} + x_{10}f_{10} + x_{11}f_{11} + x_{12}f_{12} + x_{13}f_{13} + x_{20}f_{20} + x_{21}f_{21} + x_{22}f_{22} + x_{23}f_{23} \\ y_{01} &= x_{02}f_{00} + x_{03}f_{01} + x_{04}f_{02} + x_{05}f_{03} + x_{12}f_{10} + x_{13}f_{11} + x_{14}f_{12} + x_{15}f_{13} + x_{22}f_{20} + x_{23}f_{21} + x_{24}f_{22} + x_{25}f_{23} \\ y_{10} &= x_{20}f_{00} + x_{21}f_{01} + x_{22}f_{02} + x_{23}f_{03} + x_{30}f_{10} + x_{31}f_{11} + x_{32}f_{12} + x_{33}f_{13} + x_{40}f_{20} + x_{41}f_{21} + x_{42}f_{22} + x_{43}f_{23} \\ y_{11} &= x_{22}f_{00} + x_{23}f_{01} + x_{24}f_{02} + x_{25}f_{03} + x_{32}f_{10} + x_{33}f_{11} + x_{34}f_{12} + x_{35}f_{13} + x_{42}f_{20} + x_{43}f_{21} + x_{44}f_{22} + x_{45}f_{23} \end{aligned}$$

2.

Derivative of Loss w.r.t. Filter

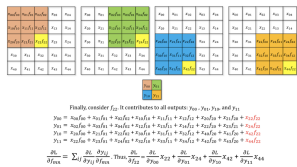
- Derivative of Loss w.r.t. f_{00}



3.

Derivative of Loss w.r.t. Filter

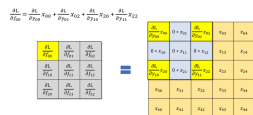
- Derivative of Loss w.r.t. f_{22}



4.

Derivative of Loss w.r.t. Filter

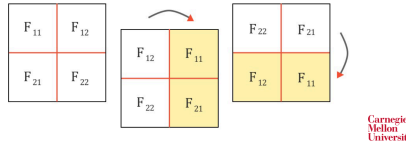
Convolution(Input, Upsampled output derivative map)



5.

Derivative of Loss w.r.t Input

- Simplifying computation:**
 - Flip the Filter horizontally and vertically



Takeaway - Stride 1 BackProp

- Both the forward and backward pass of a convolution layer can be viewed as Convolution operations

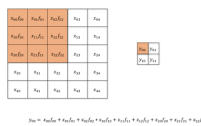
$$\frac{\partial L}{\partial F} = \text{Convolution} \left(\text{Input } X, \text{ Loss gradient } \frac{\partial L}{\partial O} \right)$$

$$\frac{\partial L}{\partial X} = \text{Convolution} \left(\text{180}^\circ \text{rotated Filter } F, \text{ Loss Gradient } \frac{\partial L}{\partial O} \right)$$

Carnegie Mellon University

Forward Pass for stride > 1

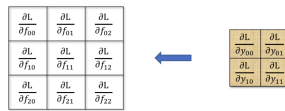
- Let us consider an example with **stride > 1** both in horizontal and vertical direction
- We have Input X and Filter as shown



Carnegie Mellon University

Derivative of Loss w.r.t. Filter

- Given the output derivative map we have to find the derivative of Loss w.r.t every Filter element

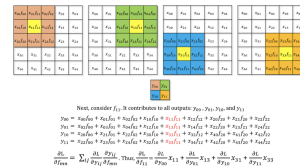


$$\frac{\partial L}{\partial f_{mn}} = \sum_{ij} \frac{\partial L}{\partial y_{ij}} \frac{\partial y_{ij}}{\partial f_{mn}}$$

Carnegie Mellon University

Derivative of Loss w.r.t. Filter

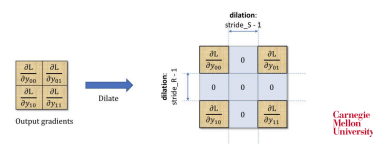
- Derivative of Loss w.r.t. f_{11}



Carnegie Mellon University

Derivative of Loss w.r.t. Filter

- To view the operation as a convolution operation we need to modify the output derivative map as follows:
 - Dilate/upsample with stride-1 zeros



Carnegie Mellon University

Derivative of Loss w.r.t. Input

- We have to find Derivative of Loss w.r.t to every input which is given as follows:

$$\frac{\partial L}{\partial x_{mn}} = \sum_{ij} \frac{\partial L}{\partial y_{ij}} \frac{\partial y_{ij}}{\partial x_{mn}}$$

Carnegie Mellon University

6.

- $$\frac{\partial L}{\partial x_{\text{res}}} = \sum_{ij} \frac{\partial L}{\partial y_{ij}} \frac{\partial y_{ij}}{\partial x_{\text{res}}}$$

Yes	Yes
Yes	Yes

Carnegie
Mellon
University

7.

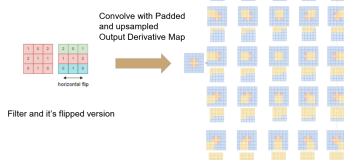
- | | | |
|----------|----------|----------|
| f_{00} | f_{01} | f_{02} |
| f_{10} | f_{11} | f_{12} |
| f_{20} | f_{21} | f_{22} |

Carnegie
Mellon
University

8.

- Carnegie
Mellon
University

9.



```
# -*- coding: utf-8 -*-
"""
Created on Fri Feb 13 17:53:49 2026

@author: reina
"""

import numpy as np
from resampling1d import *

def f1(C_out, C_in, K):
    return np.random.randint(0, 5, size=(C_out, C_in, K))

def f2(C_out):
    return np.random.randint(0, 3, size=(C_out, 1))

class Conv1d_stride1:
    def __init__(
        self, in_channels, out_channels, kernel_size, weight_init_fn, bias_init_fn
    ):
        self.C_in = in_channels
        self.C_out = out_channels
        self.K = kernel_size

        if weight_init_fn is None:
            self.W = np.random.normal(0, 1.0, (out_channels, in_channels, kernel_size))
        else:
            self.W = weight_init_fn(out_channels, in_channels, kernel_size)
        if bias_init_fn is None:
            self.b = np.zeros(shape=(out_channels, 1))
        else:
            self.b = bias_init_fn(out_channels)
        self.dLdW = None
        self.dLdb = None

    def forward(self, x):
        self.x = x # save input for backprop
        N, C_in, W_in = x.shape
        W_out = (W_in - self.K) + 1
```

```

z = np.empty(shape=(N, self.C_out, W_out), dtype=np.float64)
for i in range(W_out):
    z[:, :, i] = np.tensordot(
        x[:, :, i : i + self.K], self.W, axes=((1, 2), (1, 2))
    ) + self.b.reshape(-1)
return z

def backward(self, dLdz):
    N, C_out, W_out = dLdz.shape
    N, C_in, W_in = self.x.shape
    assert C_out == self.C_out and C_in == self.C_in
    # find dLdW
    self.dLdW = np.empty(shape=(C_out, C_in, self.K), dtype=np.float64)
    # for i in range(C_out):
    #     # Convolve input x with dLdz as filter.
    #     for j in range(self.K):
    #         # With tensordot, broadcasting is handled implicitly.
    #         # Size of immediate result after convolution is (C_in, 1) reshape to (C_in,).
    #         self.dLdW[i, :, j] = np.tensordot(self.x[:, :, j:j+W_out], dLdz[:, i:i+1, :],
    #             axes=((0,2),(0,2))).reshape(-1)
    for i in range(self.K):
        self.dLdW[:, :, i] = np.tensordot(
            self.x[:, :, i : i + W_out], dLdz, axes=((0, 2), (0, 2))
        ).transpose(1, 0)

    # find dLdb
    self.dLdb = np.sum(dLdz, axis=(0, 2))
    # find dLdx
    dLdx = np.zeros(shape=self.x.shape, dtype=np.float64)
    # pad dLdz with K-1 zeros left and right
    # dLdz size: before: W_in-K+1, after: W_in-K+1+2(K-1) = W_in+K-1
    # i.e. to get back original input size, W_in, after convolving dLdz with 180° rotated kernel as filter.
    dLdz = np.pad(
        dLdz, pad_width=((0, 0), (0, 0), (self.K - 1, self.K - 1)), mode="constant"
    )

    # for i in range(C_out):
    #     # Convolve dLdz with 180° rotated kernel as filter.
    #     for j in range(W_in):
    #         # because all outputs are of the same size (N, C_in) we should accumulate the results.
    #         dLdx[:, :, j] += np.tensordot(dLdz[:, i:i+1, j:j+self.K], np.flip(self.W[i:i+1, :, :],
    #             axis=2), axes=((1,2),(0,2))))

    for i in range(W_in):
        dLdx[:, :, i] = np.tensordot(
            dLdz[:, :, i : i + self.K],
            np.flip(self.W, axis=2),
            axes=((1, 2), (0, 2)),
        )
    return dLdx

class Conv1d:
    def __init__(
        self,
        in_channels,
        out_channels,
        kernel_size,
        stride,
        padding=0,
        weight_init_fn=None,
        bias_init_fn=None,
    ):
        self.stride = stride
        self.padding = padding
        self.C_in = in_channels
        self.C_out = out_channels
        self.K = kernel_size
        self.conv1d_stride1 = Conv1d_stride1(
            self.C_in, self.C_out, self.K, weight_init_fn, bias_init_fn
        )
        self.downsampled1d = Downsample1d(downsampling_factor=stride)

    def forward(self, x):
        # pad with zeros
        x = np.pad(
            x, pad_width=((0, 0), (0, 0), (self.padding, self.padding)), mode="constant"
        )
        # Conv1d forward
        z = self.conv1d_stride1.forward(x)
        # Downsample1d forward
        z = self.downsampled1d.forward(z)
        return z

    def backward(self, dLdz):
        # Downsample1d backward
        dLdx = self.downsampled1d.backward(dLdz)
        # Conv1d backward
        dLdx = self.conv1d_stride1.backward(dLdx)

```

```

        # unpad zeros
        if self.padding > 0:
            dLdx = dLdx[:, :, self.padding : -self.padding]
        return dLdx

if __name__ == "__main__":
    N = 10
    C_in = 5
    C_out = 8
    K = 3
    W_in = 5

    conv1d_stride1 = Conv1d_stride1(C_in, C_out, K, f1, f2)
    conv1d = Conv1d(
        C_in, C_out, K, stride=2, padding=0, weight_init_fn=f1, bias_init_fn=f2
    )

    # forward
    x = np.random.randint(0, 10, size=(N, C_in, W_in))
    z = conv1d.forward(x)

    # backward
    dLdz = np.random.random(z.shape)
    dLdx = conv1d.backward(dLdz)

    # print(h)
    print("x.shape", x.shape)
    print("z.shape ", z.shape)
    print("dLdx.shape ", dLdx.shape)
    print("dLdz.shape", dLdz.shape)

```

14. Code: Conv2d.py

```

# -*- coding: utf-8 -*-
"""
Created on Fri Feb 13 18:03:49 2026

@author: reina
"""

import numpy as np
from resampling2d import *

def f1(C_out, C_in, K1, K2):
    return np.random.randint(0, 5, size=(C_out, C_in, K1, K2))

def f2(C_out):
    return np.random.randint(0, 3, size=(C_out, 1))

class Conv2d_stride1:
    def __init__(
        self, in_channels, out_channels, kernel_size, weight_init_fn, bias_init_fn
    ):
        self.C_in = in_channels
        self.C_out = out_channels
        self.K = kernel_size

        self.W = weight_init_fn(out_channels, in_channels, kernel_size, kernel_size)
        self.b = bias_init_fn(out_channels)

        self.dLdW = None
        self.dLdb = None

    def forward(self, x):
        self.x = x
        N, C_in, H_in, W_in = x.shape
        H_out = H_in - self.K + 1
        W_out = W_in - self.K + 1
        z = np.empty(shape=(N, self.C_out, H_out, W_out), dtype=np.float64)
        for i in range(H_out):
            for j in range(W_out):
                z[:, :, i, j] = np.tensordot(
                    x[:, :, i : i + self.K, j : j + self.K],
                    self.W,
                    axes=((1, 2, 3), (1, 2, 3)),
                ) + self.b.reshape(-1)
        return z

    def backward(self, dLdz):
        N, C_out, H_out, W_out = dLdz.shape
        N, C_in, H_in, W_in = self.x.shape
        assert C_out == self.C_out and C_in == self.C_in
        # find dLdW
        self.dLdW = np.empty(shape=(C_out, C_in, self.K, self.K), dtype=np.float64)
        # for i in range(C_out):
        #     for j in range(self.K):
        #         for k in range(self.K):

```

```

#         self.dLdW[i, :, j, k] = np.tensordot(self.x[:, :, j:j+H_out, k:k+W_out],
#         (dLdz[:, i, :, :])[:, np.newaxis, :, :],
#         axes=((0,2,3),(0,2,3))).reshape(-1)
for i in range(self.K):
    for j in range(self.K):
        self.dLdW[:, :, i, j] = np.tensordot(
            self.x[:, :, i : i + H_out, j : j + W_out],
            dLdz,
            axes=((0, 2, 3), (0, 2, 3)),
        ).transpose(1, 0)

# find dLdb
self.dLdb = np.sum(dLdz, axis=(0, 2, 3))
# find dLdx
dLdx = np.zeros(shape=self.x.shape, dtype=np.float64)
dLdz = np.pad(
    dLdz,
    pad_width=(
        (0, 0),
        (0, 0),
        (self.K - 1, self.K - 1),
        (self.K - 1, self.K - 1),
    ),
    mode="constant",
)
# for i in range(C_out):
#     for j in range(H_in):
#         for k in range(W_in):
#             dLdx[:, :, j, k] += np.tensordot((dLdz[:, i, j:j+self.K, k:k+self.K])[:, np.newaxis, :, :],
#             np.flip((self.W[i, :, :, :])[np.newaxis, :, :, :],
#             axis=(2,3)), axes=((1,2,3),(0,2,3)))
for i in range(H_in):
    for j in range(W_in):
        dLdx[:, :, i, j] = np.tensordot(
            dLdz[:, :, i : i + self.K, j : j + self.K],
            np.flip(self.W, axis=(2, 3)),
            axes=((1, 2, 3), (0, 2, 3)),
        )
return dLdx

class Conv2d:
    def __init__(
        self,
        in_channels,
        out_channels,
        kernel_size,
        stride,
        padding=0,
        weight_init_fn=None,
        bias_init_fn=None,
    ):
        self.stride = stride
        self.padding = padding
        self.C_in = in_channels
        self.C_out = out_channels
        self.K = kernel_size
        self.conv2d_stride1 = Conv2d_stride1(
            self.C_in, self.C_out, self.K, weight_init_fn, bias_init_fn
        )
        self.downsample2d = Downsample2d(downsampling_factor=stride)

    def forward(self, x):
        # pad with zeros
        x = np.pad(
            x,
            pad_width=(
                (0, 0),
                (0, 0),
                (self.padding, self.padding),
                (self.padding, self.padding),
            ),
            mode="constant",
        )
        # Conv2d forward
        z = self.conv2d_stride1.forward(x)
        # Downsample forward
        z = self.downsample2d.forward(z)
        return z

    def backward(self, dLdz):
        # Downsample backward
        dLdx = self.downsample2d.backward(dLdz)
        # Conv2d backward
        dLdx = self.conv2d_stride1.backward(dLdx)
        # unpad zeros
        if self.padding > 0:
            dLdx = dLdx[
                :, :, self.padding : -self.padding, self.padding : -self.padding
            ]

```

```

        return dLdx

if __name__ == "__main__":
    # for testing purposes
    N = 10
    C_in = 5
    C_out = 8
    K = 3
    H_in = 5
    W_in = 5

    conv2d_stride1 = Conv2d_stride1(C_in, C_out, K, f1, f2)
    conv2d = Conv2d(C_in, C_out, K, 2, 0, f1, f2)

    # forward
    x = np.random.randint(0, 10, (N, C_in, H_in, W_in))
    z = conv2d_stride1.forward(x)
    # backward
    dLdz = np.random.randint(0, 10, size=z.shape)
    dLdx = conv2d_stride1.backward(dLdz)

    print("x.shape ", x.shape)
    print("z.shape ", z.shape)

    # print("dLdx ", dLdx)
    print("dLdx.shape ", dLdx.shape)
    print("dLdz.shape ", dLdz.shape)

    # forward
    x = np.random.randint(0, 10, (N, C_in, H_in, W_in))
    z1 = conv2d.forward(x)
    # backward
    dLdz1 = np.random.randint(0, 10, size=z1.shape)
    dLdx1 = conv2d.backward(dLdz1)

    print("x.shape ", x.shape)
    print("z1.shape ", z1.shape)
    print("dLdx1.shape ", dLdx1.shape)
    print("dLdz1.shape ", dLdz1.shape)

```

15. im2col

1. As shown on previous source code, if we do not use `np.tensordot` we will use a lot of for-loops to implement the convolutions, an alternative to this is using `im2col`. On this section we will learn how to implement convolutions on a vectorized fashion.
2. If we expand all possible windows on memory and perform the dot product as a matrix multiplication, we'll get 200x or more speedups, at the expense of more memory consumption.
3. For example, if the input is $[227 \times 227 \times 3]$ and it is to be convolved with $11 \times 11 \times 3$ filters at stride 4 and padding 0, then we would take $[11 \times 11 \times 3]$ blocks of pixels in the input and stretch each block into a column vector of size $11 \times 11 \times 3 = 363$.
4. Calculating with input 227 with stride 4 and padding 0, gives $((227-11)/4)+1 = 55$ locations along both width and height, leading to an output matrix X_{col} of size $[363 \times 3025]$, Note: $55 * 55 = 3025$.
5. Here every column is a stretched out receptive field (patch with depth) and there are $55 * 55 = 3025$ of them in total.
6. The weights of the CONV layer are similarly stretched out into rows. For example, if there are 96 filters of size $[11 \times 11 \times 3]$ this would give a matrix W_{row} of size $[96 \times 363]$, where $11 \times 11 \times 3 = 363$
7. After the image and the kernel are converted, the convolution can be implemented as a simple matrix multiplication, in our case it will be W_{col} $[96 \times 363]$ multiplied by X_{col} $[363 \times 3025]$ resulting as a matrix $[96 \times 3025]$, that need to be reshaped back to $[55 \times 55 \times 96]$.
8. This final reshape can also be implemented as a function called `col2im`.
9. Notice that some implementations of `im2col` will have this result **transposed**, if this is the case then the order of the matrix multiplication must be changed.
10. To summarize how we calculate the `im2col` output sizes:

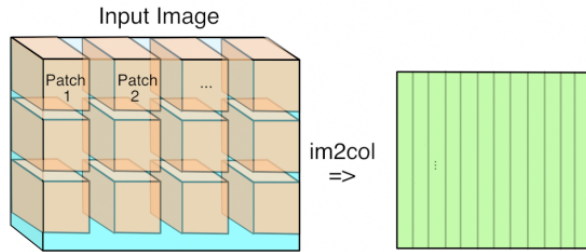
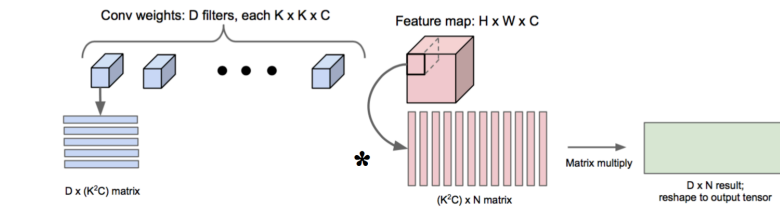
```

[img_height, img_width, img_channels] = size(img);
newImgHeight = floor(((img_height + 2*P - ksize) / S)+1);
newImgWidth = floor(((img_width + 2*P - ksize) / S)+1);
cols = single(zeros((img_channels*ksize*ksize),(newImgHeight * newImgWidth)));

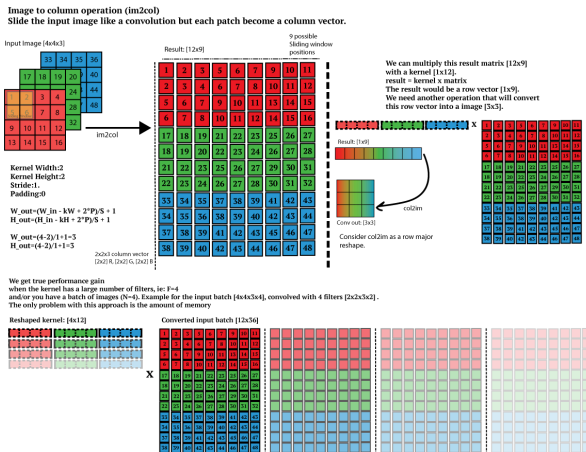
```

11. im2col (transposed version)

1. Reshape image to be convolved to shape $(H_{out} * W_{out}, K^2 C)$, and reshape kernel to shape $(K^2 C_{in}, C_{out})$.
2. $(H_{out} * W_{out}, K^2 C_{in}) \cdot (K^2 C_{in}, C_{out}) = (H_{out} * W_{out}, C_{out})$
3. In standard convolution we aggregate across filter size, K , and channels, C . Hence, why reshaping the image tensor and the filters into of size $K^2 C$ in one of its two dimensions.



12.



13.

16. Point-wise (1x1) Convolution

1. pointwise 1x1 convolutions are often used to increase or decrease the number of channels. i.e. $(N, C1, H, W)$ to $(N, C2, H, W)$, with $C1C2$.
2. Can also be mimicked using nn.Linear: Feature map: $(N, C1, H, W)$ - $\text{rarrange} \rightarrow (N, H, W, C1)$ -Linear- $\rightarrow (N, H, W, C2)$.

17. Depth-wise Convolution

18. Adaptive Pooling (SPPNet)

1. kernel window_size: $\text{ceil}(\frac{\text{feature_map_size}}{\text{pooling_size}})$
2. stride: $\text{floor}(\frac{\text{feature_map_size}}{\text{pooling_size}})$
3. for example: Feature map size is 13×13 ,
 1. to get $\text{Pool}_{3 \times 3}$:
 1. kernel_size = $\text{ceil}(\frac{13}{3}) = 5$
 2. stride = $\text{floor}(\frac{13}{4}) = 4$
 3. $\frac{W-K}{S} + 1 = \frac{13-5}{4} + 1 = 3$
 2. to get $\text{Pool}_{2 \times 2}$:
 1. kernel_size = $\text{ceil}(\frac{13}{2}) = 7$
 2. stride = $\text{floor}(\frac{13}{2}) = 6$
 3. $\frac{W-K}{S} + 1 = \frac{13-7}{6} + 1 = 2$
 3. to get $\text{Pool}_{1 \times 1}$:
 1. kernel_size = $\text{ceil}(\frac{13}{1}) = 13$
 2. stride = $\text{floor}(\frac{13}{1}) = 13$
 3. $\frac{W-K}{S} + 1 = \frac{13-13}{13} + 1 = 1$

Deformable CNNs

1. Formal definition:

1. Standard convolution

$$\mathcal{R} = (-1, -1), (-1, 0), \dots, (0, 1), (1, 1)$$

$$y(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n),$$

2.

3. Recall the above formula's similarity with BFS.

4. Deformable convolution

5.

$$\mathcal{R} = (-1, -1), (-1, 0), \dots, (0, 1), (1, 1)$$
$$\mathbf{y}(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n + \Delta \mathbf{p}_n),$$

6. where $\Delta \mathbf{p}_n$ is the offsets for sampling.

0	0	0	0	0	0
0	1	2	1	0	0
0	0	1	2	3	4
0	1	5	6	7	8
0	0	9	10	11	12
0	0	13	14	15	16

Standard Convolution

CIS 680



0	1	0	0	1	0
0	0	1	1	0	0
0	0	1	2	3	4
0	1	5	6	7	8
0	0	9	10	11	12
0	0	13	14	15	16

Deformable Convolution

7.

8. Deformable convolution increases receptive field compared to standard convolution.

Related: [Resampling](#)

References:

1. https://velog.io/@tobigs1516_image/FCN-Fully-Convolution-Network-for-semantic-segmentation-review
2. https://leonardoaraujosantos.gitbook.io/artificial-intelligence/machine_learning/deep_learning/image_segmentation
3. Backpropagation in CNN, 11785 Introduction to Deep Learning (Fall 2022), Recitation 5, Ruoyu Hua.
4. Spatial Transformer Networks and Deformable CNNs, CIS 680